You Could Have Invented Fenwick Trees!

Brent Yorgey

21 May 2021

About me

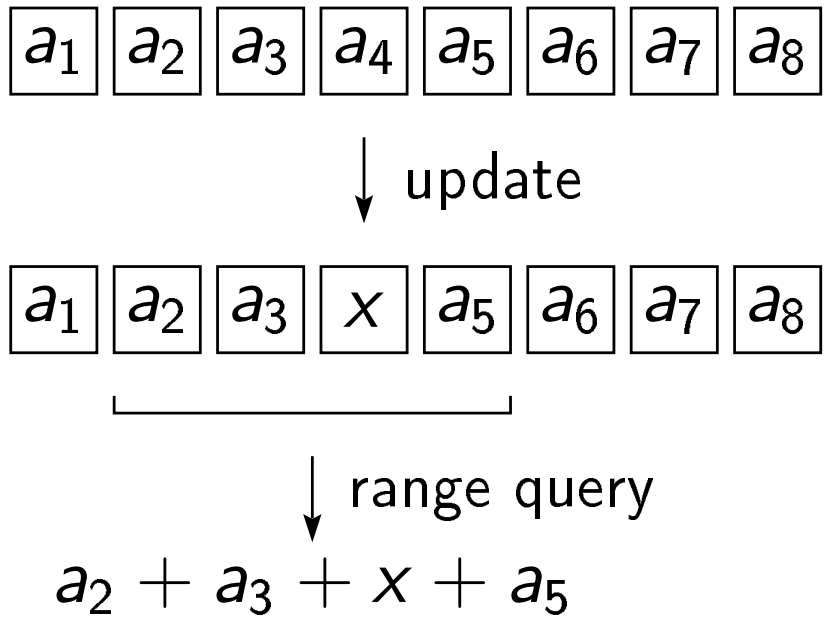
(2008–2014)

(2015–)

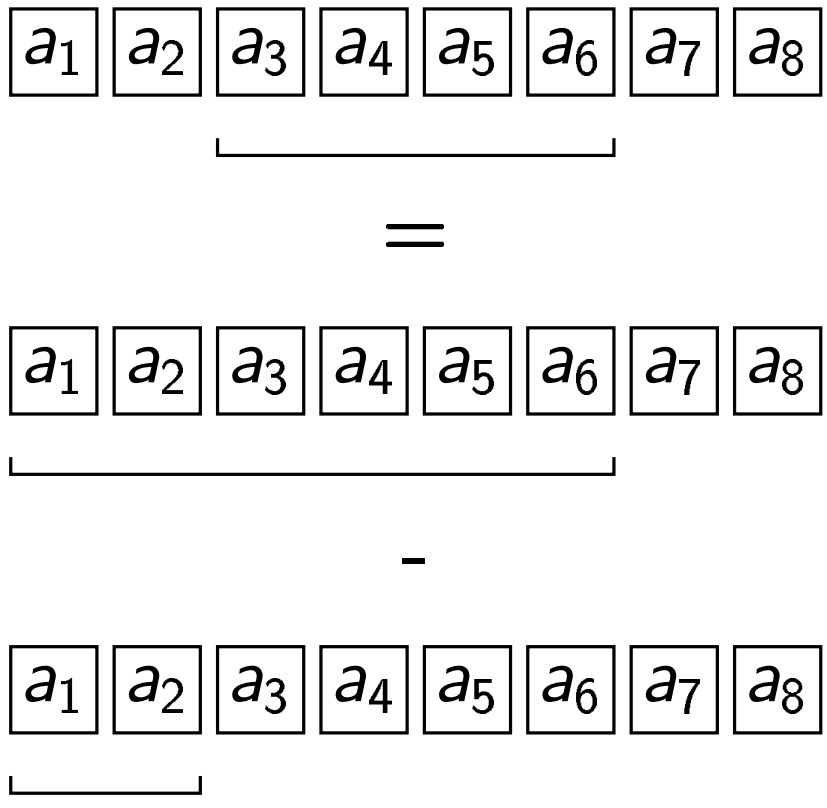
Things I Like (a very small sample):
FP, EDSLs, combinatorics, category theory, teaching,
competitive programming, questions from the audience

Introduction

Sequence operations



Prefix queries



Solutions

approach	update	prefix query
	$O(1)$	$O(n)$
	$O(n)$	$O(1)$
	$O(\lg n)$	$O(\lg n)$

Solutions

approach	update	prefix query
just store sequence	$O(1)$	$O(n)$
	$O(n)$	$O(1)$
	$O(\lg n)$	$O(\lg n)$

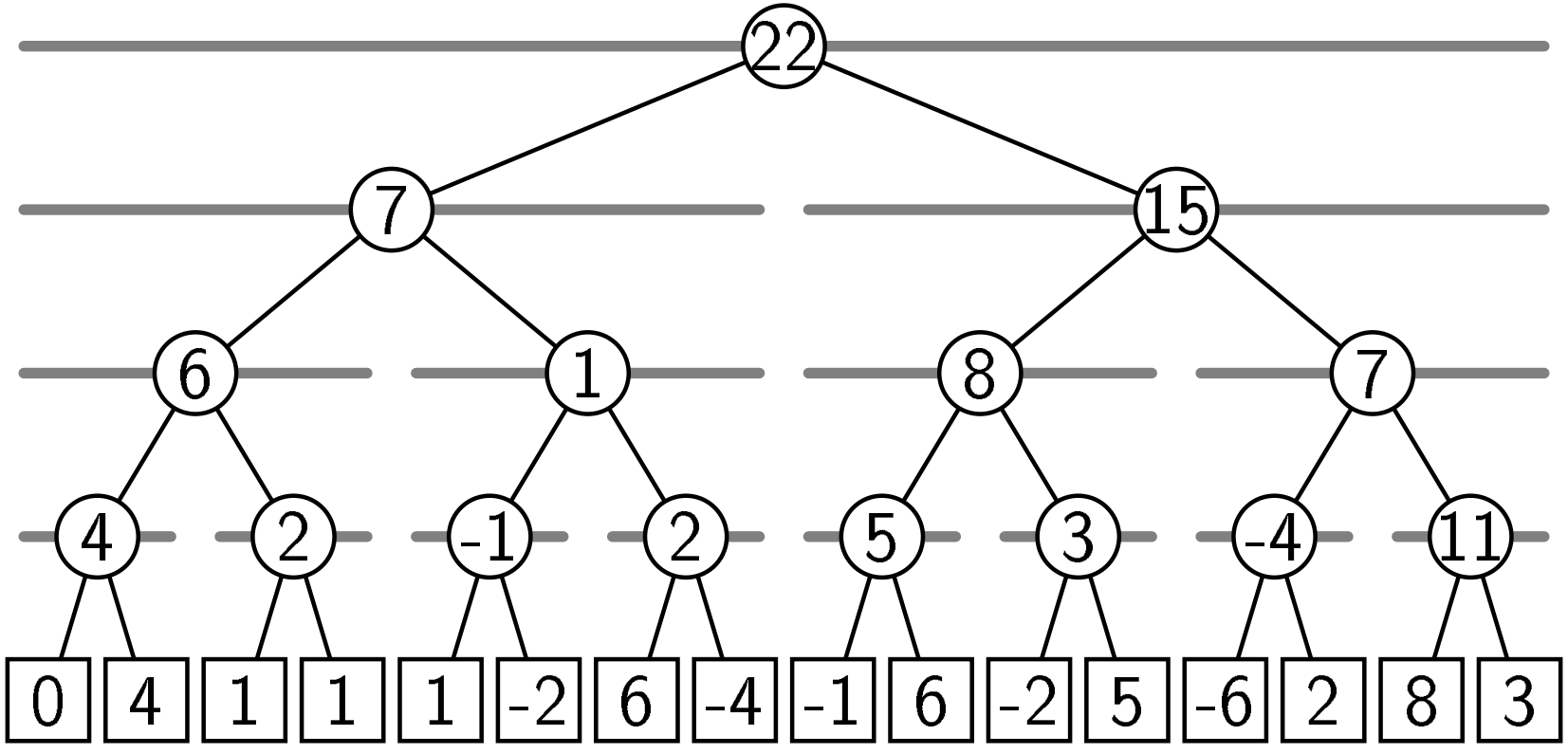
Solutions

approach	update	prefix query
just store sequence	$O(1)$	$O(n)$
cache prefix sums	$O(n)$	$O(1)$
	$O(\lg n)$	$O(\lg n)$

Solutions

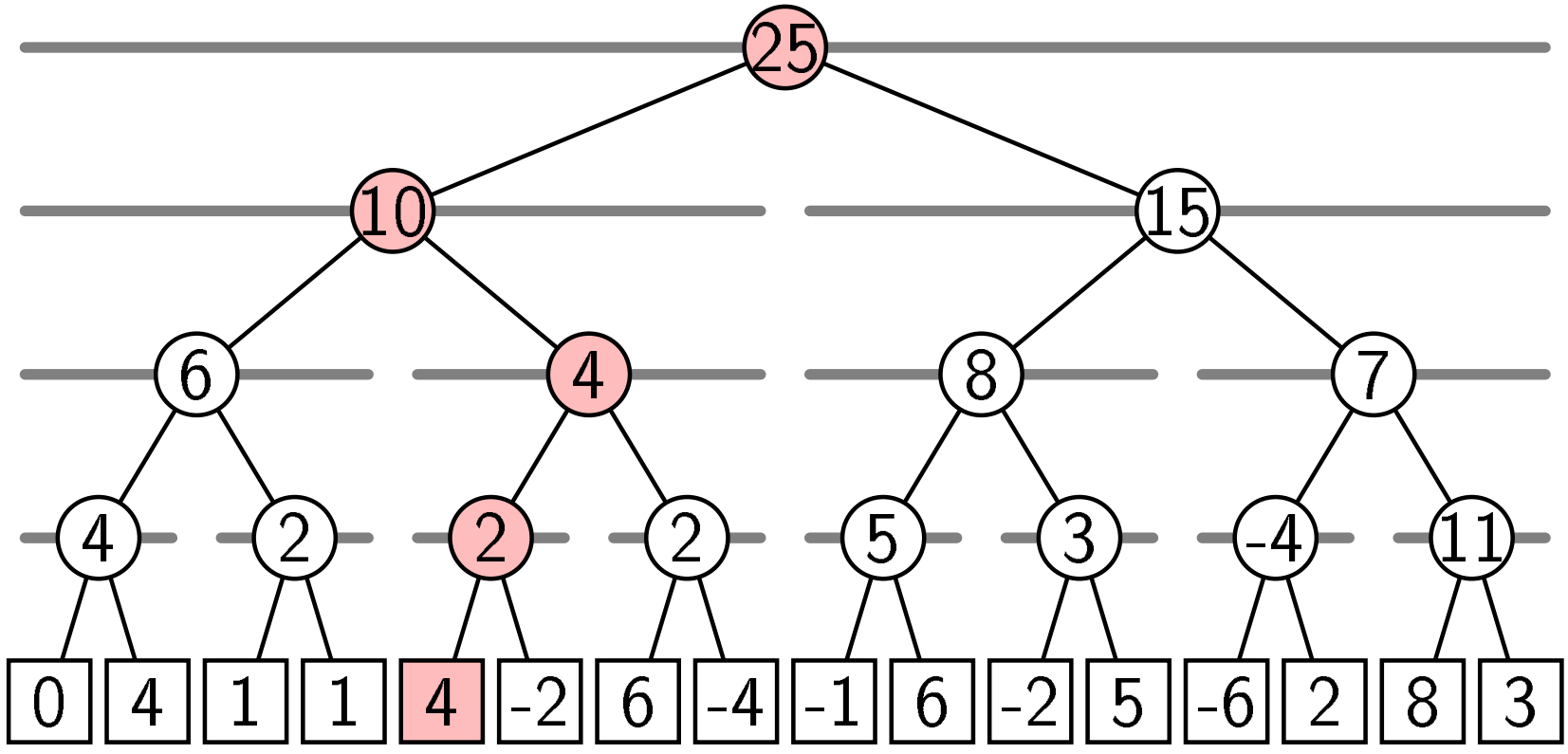
approach	update	prefix query
just store sequence	$O(1)$	$O(n)$
cache prefix sums	$O(n)$	$O(1)$
cache more cleverly. . . ?	$O(\lg n)$	$O(\lg n)$

Segment trees

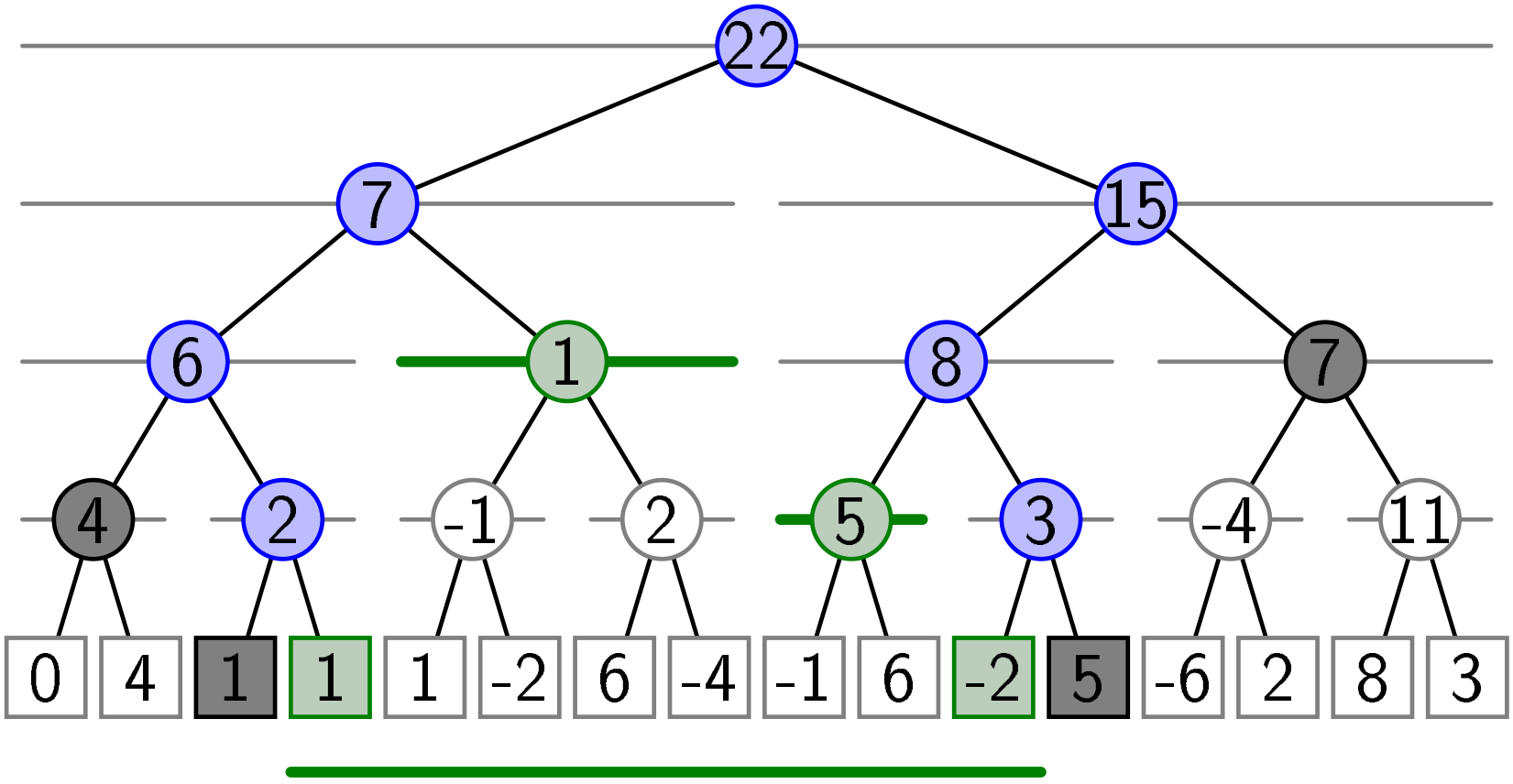


(assume $n = 2^k$)

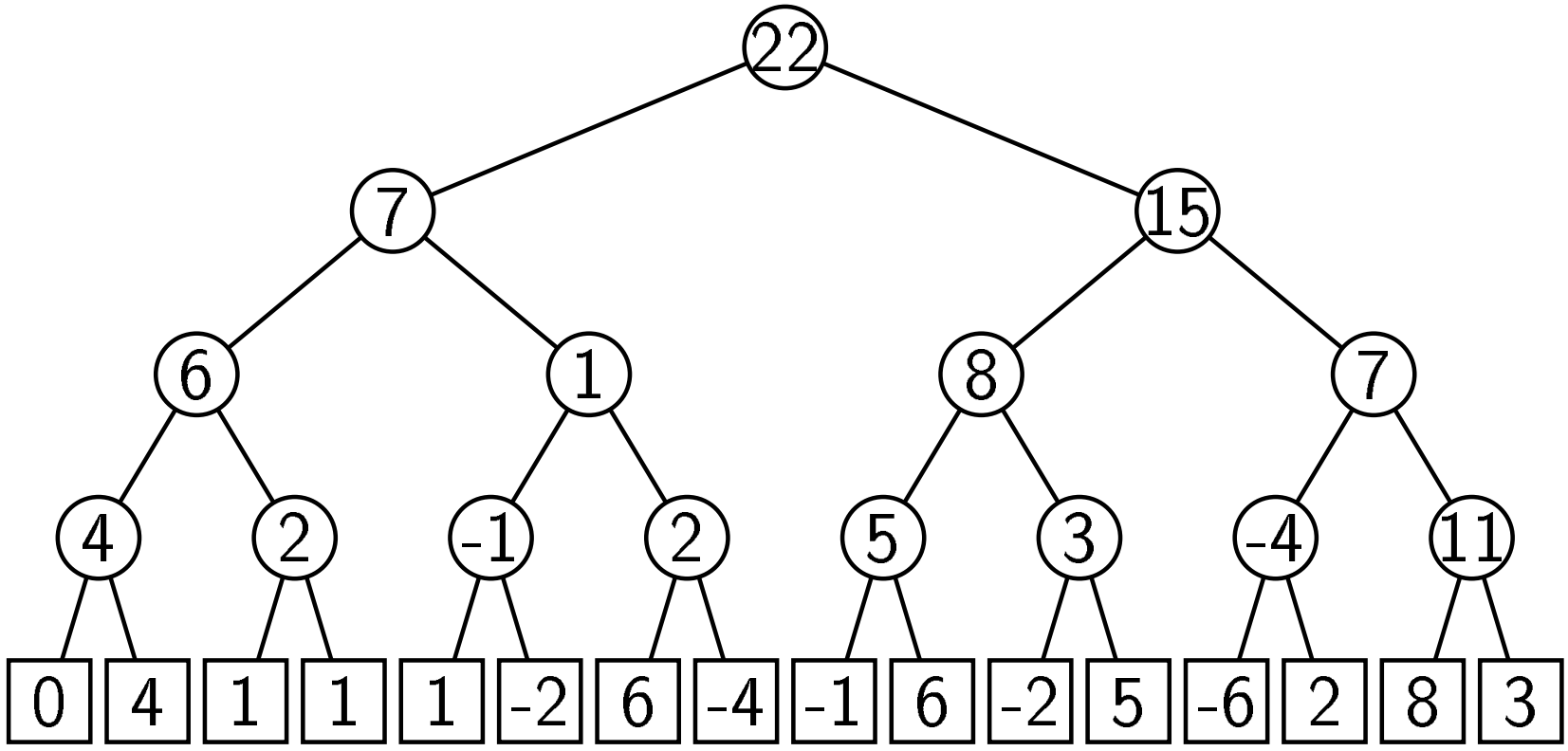
Updating a segment tree



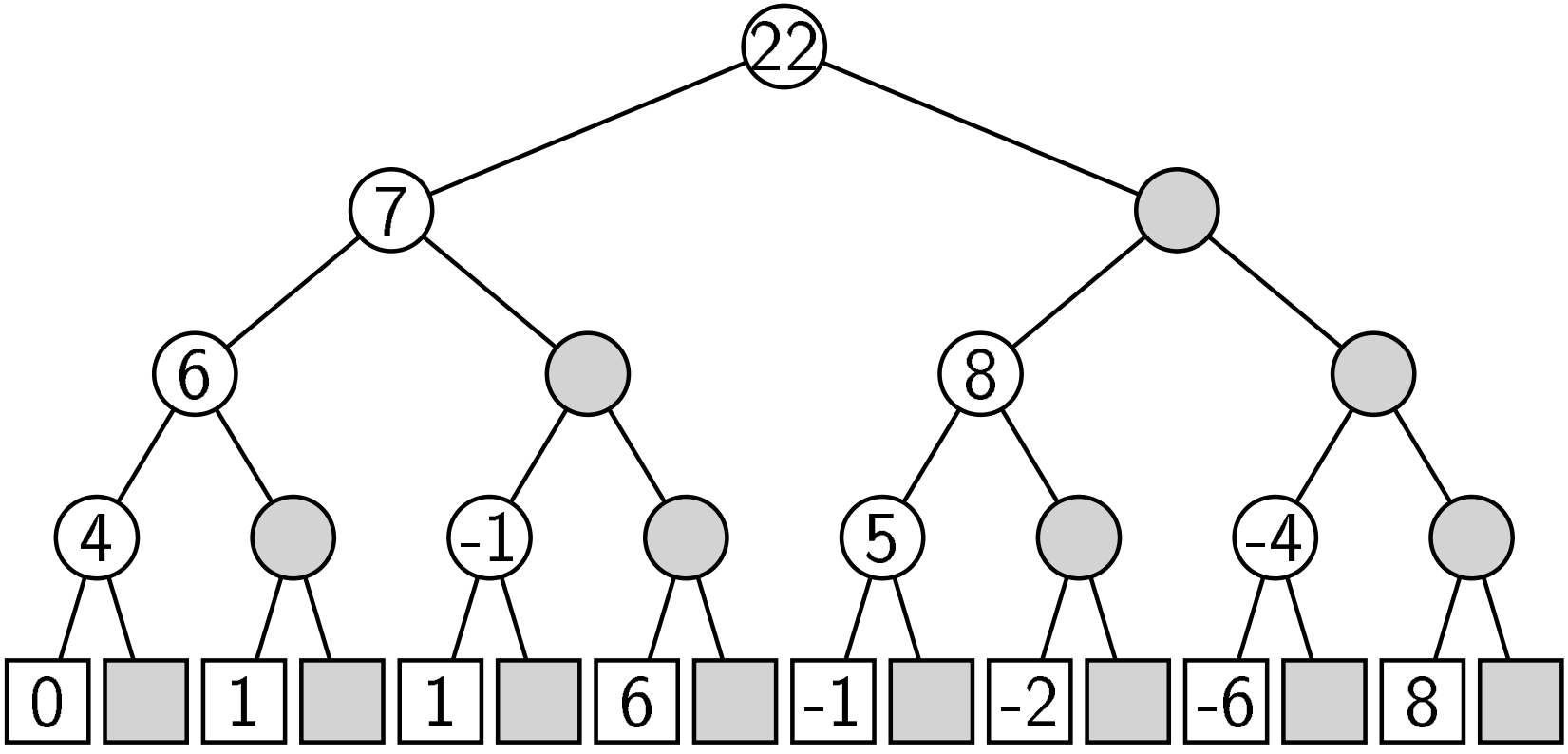
Range query on a segment tree



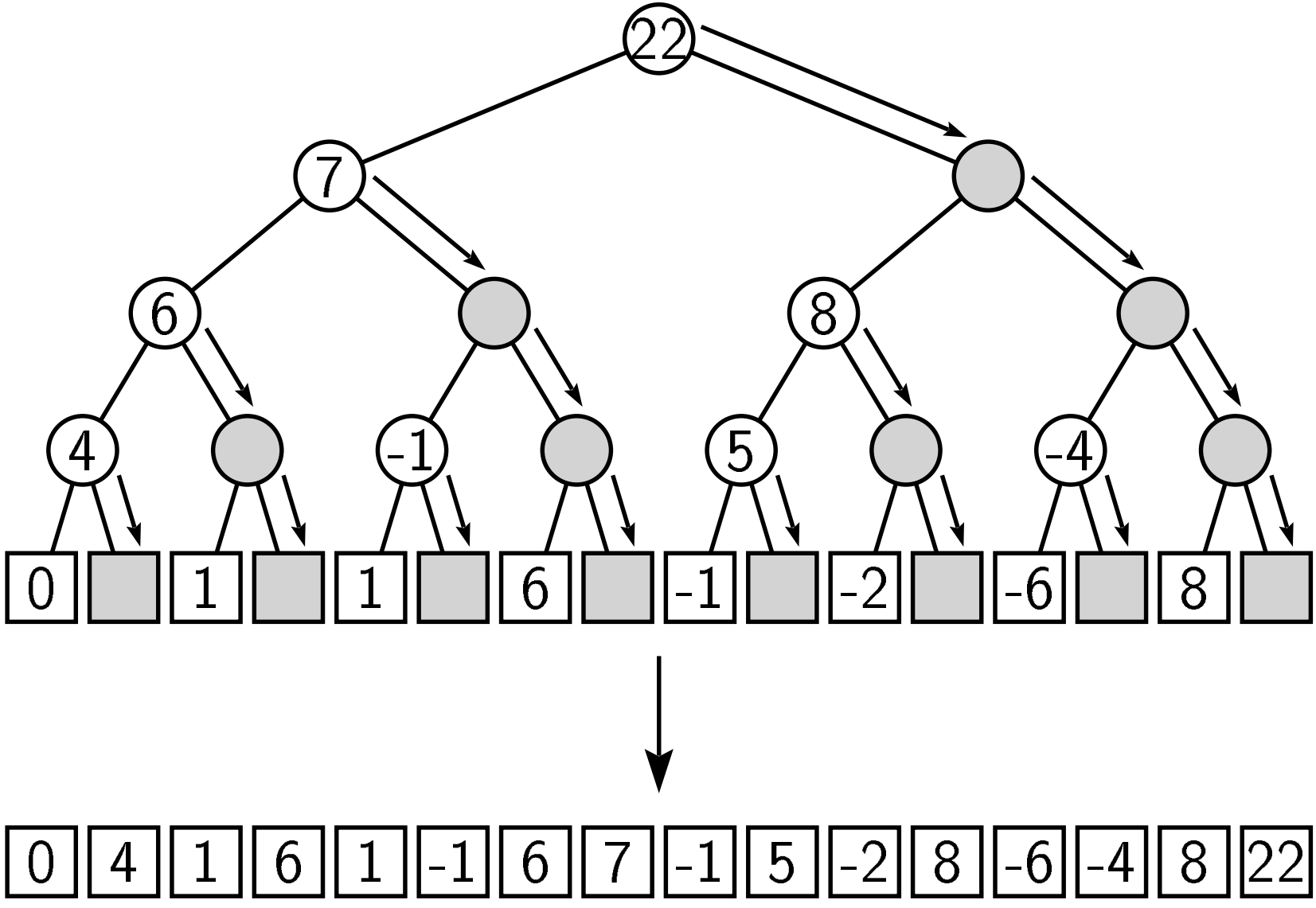
Thinning segment trees



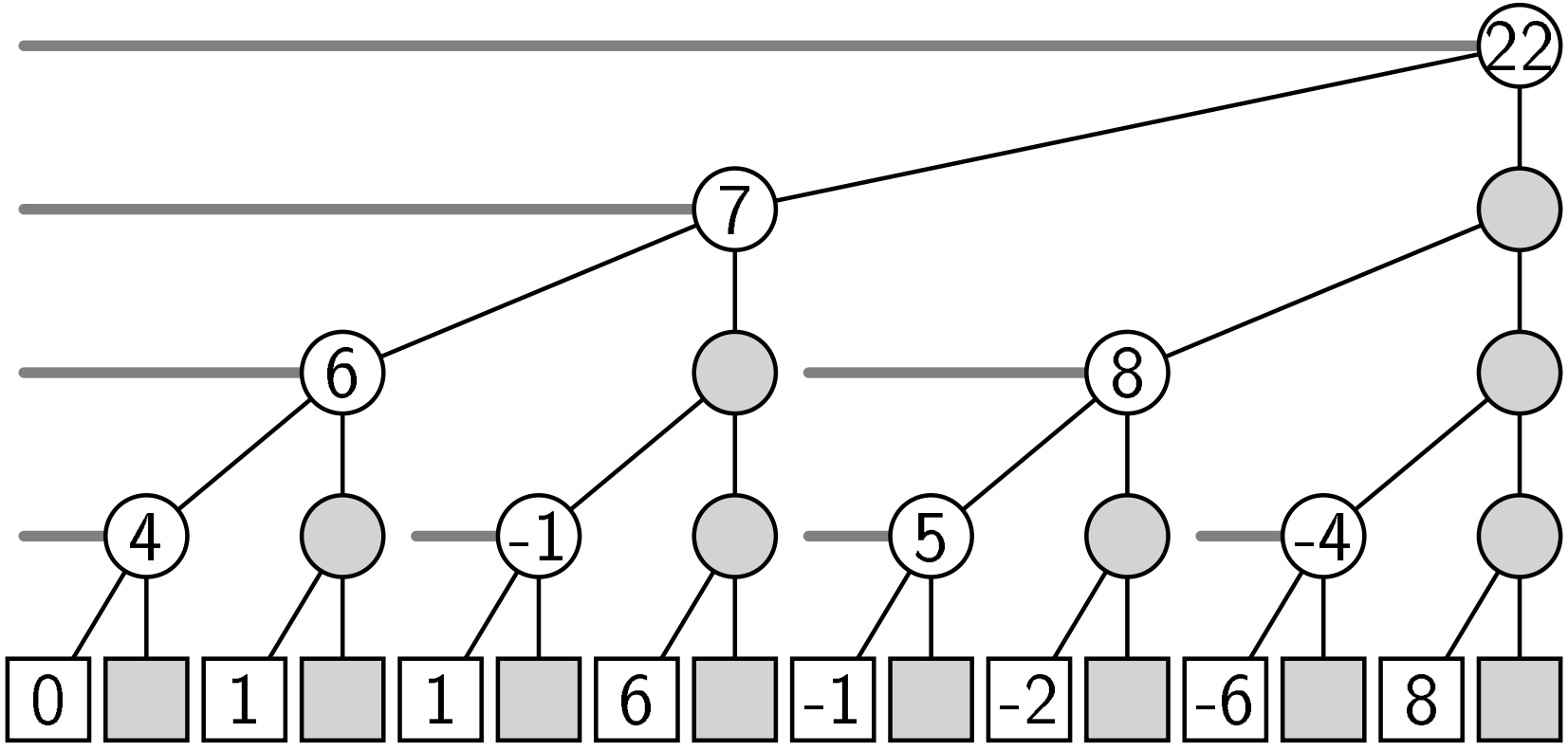
Thinning segment trees



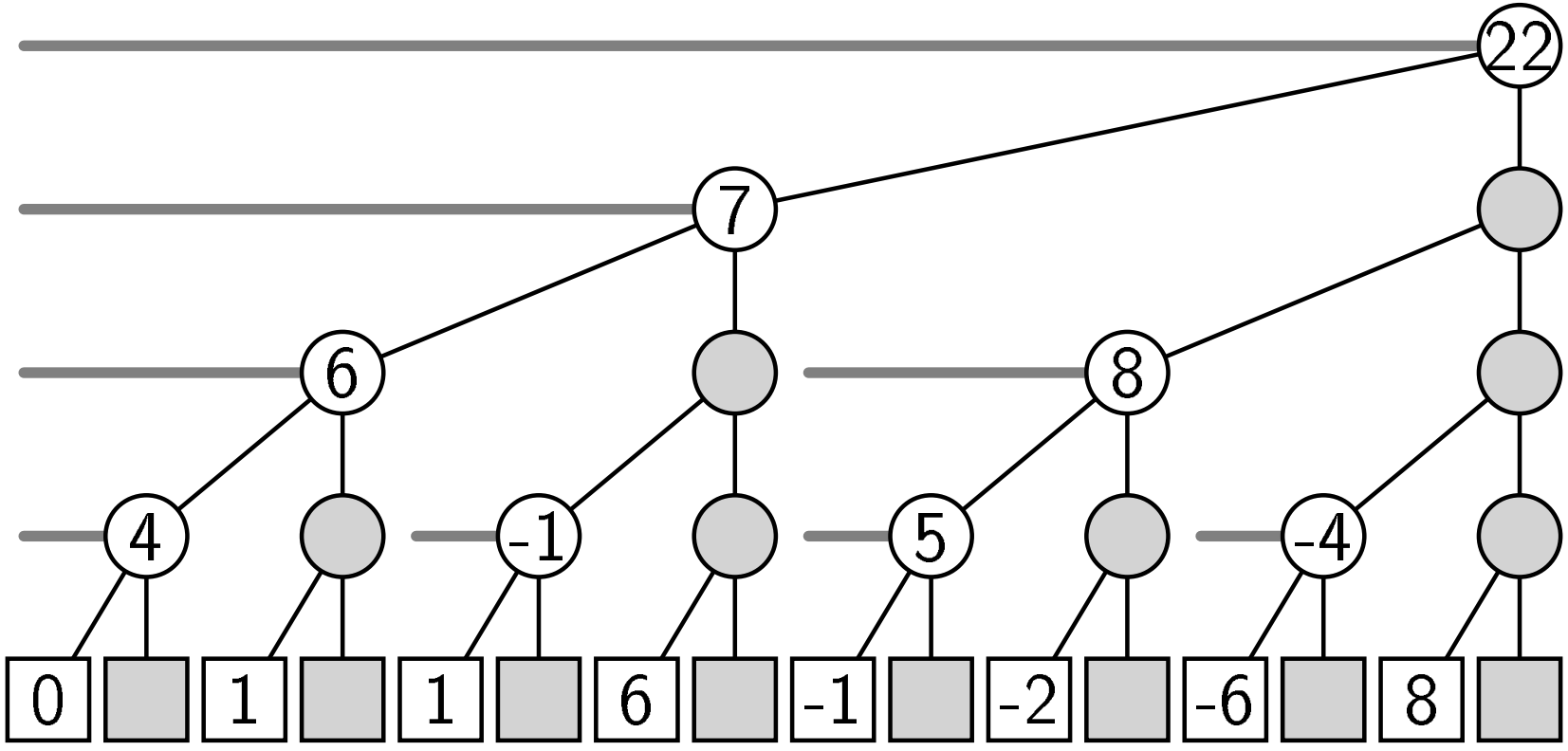
Storing a thinned segment tree



Fenwick tree = right-leaning, thinned segment tree



Fenwick tree = right-leaning, thinned segment tree



. . . but how to move around?

Fenwick trees

SOFTWARE—PRACTICE AND EXPERIENCE, VOL. 24(3), 327–336 (MARCH 1994)

A New Data Structure for Cumulative Frequency Tables

peter m. fenwick
Department of Computer Science, University of Auckland, Private Bag 92019, Auckland, New Zealand (email: p_fenwick@cs.auckland.ac.nz)

SUMMARY

A new method (the 'binary indexed tree') is presented for maintaining the cumulative frequencies which are needed to support dynamic arithmetic data compression. It is based on a decomposition of the cumulative frequencies into portions which parallel the binary representation of the index of the table element (or symbol). The operations to traverse the data structure are based on the binary coding of the index. In comparison with previous methods, the binary indexed tree is faster, using more compact data and simpler code. The access time for all operations is either constant or proportional to the logarithm of the table size. In conjunction with the compact data structure, this makes the new method particularly suitable for large symbol alphabets.

key words: Binary indexed tree Arithmetic coding Cumulative frequencies

INTRODUCTION

A major cost in adaptive arithmetic data compression is the maintenance of the table of cumulative frequencies which is needed in reducing the range for successive symbols. Witten, Neal and Cleary[1] ease the problem by providing a move-to-front mapping of the symbols which ensures that the most frequent symbols are kept near the front of the search space. It works well for highly skewed alphabets (which may be expected to compress well) but is much less efficient for more uniform distributions of symbol frequency. Moffat[2] describes a tree structure (actually a heap) which provides a linear-time access to all symbols. Jones[3] uses splay trees to provide an optimized data structure for handling the frequency tables. The three techniques will be referred to in this paper as MTF, HEAP and SPLAY, respectively. In all cases they attempt to keep frequently used symbols in quickly-referenced positions within the data structure, but at the cost of sometimes extensive data reorganization.

This current paper describes a new method which uses only a single array to store the frequencies, but stores them in a carefully chosen pattern to suit a novel search technique whose cost is proportional to the number of 1 bits in the element index. This cost applies to both updating and interrogating the table. In comparison with the other methods it is simple, compact and fast and involves no reorganization or movement of the data.

CCC 0038–0644/94/030327–10

Received 7 June 1993
Revised 6 October 1993

Peter Fenwick, 1994
A New Data Structure for Cumulative Frequency Tables

Implementing Fenwick trees

```
class FenwickTree {
    private long[] a;
    public FenwickTree(int n) { a = new long[n+1]; }
    public long prefix(int i) {
        long s = 0;
        for (; i > 0; i -= LSB(i)) s += a[i]; return s;
    }
    public void update(int i, long delta) {
        for (; i < a.length; i += LSB(i)) a[i] += delta;
    }
    public long range(int i, int j) {
        return prefix(j) - prefix(i-1);
    }
    public long get(int i) { return range(i,i); }
    private int LSB(int i) { return i & (-i); }
}
```

LSB = Least Significant Bit

$$\mathrm{LSB}(01101100) = 00000100$$

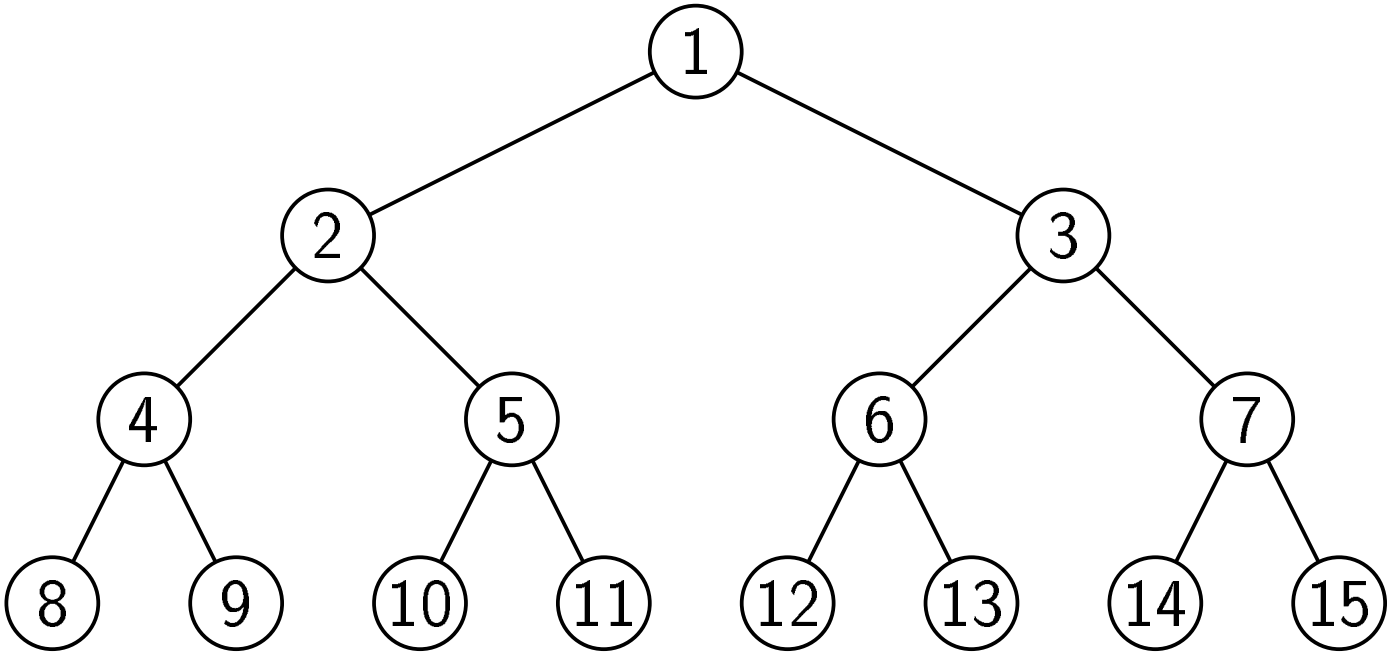
LSB = Least Significant Bit

$$\mathrm{LSB}(01101100) = 00000100$$

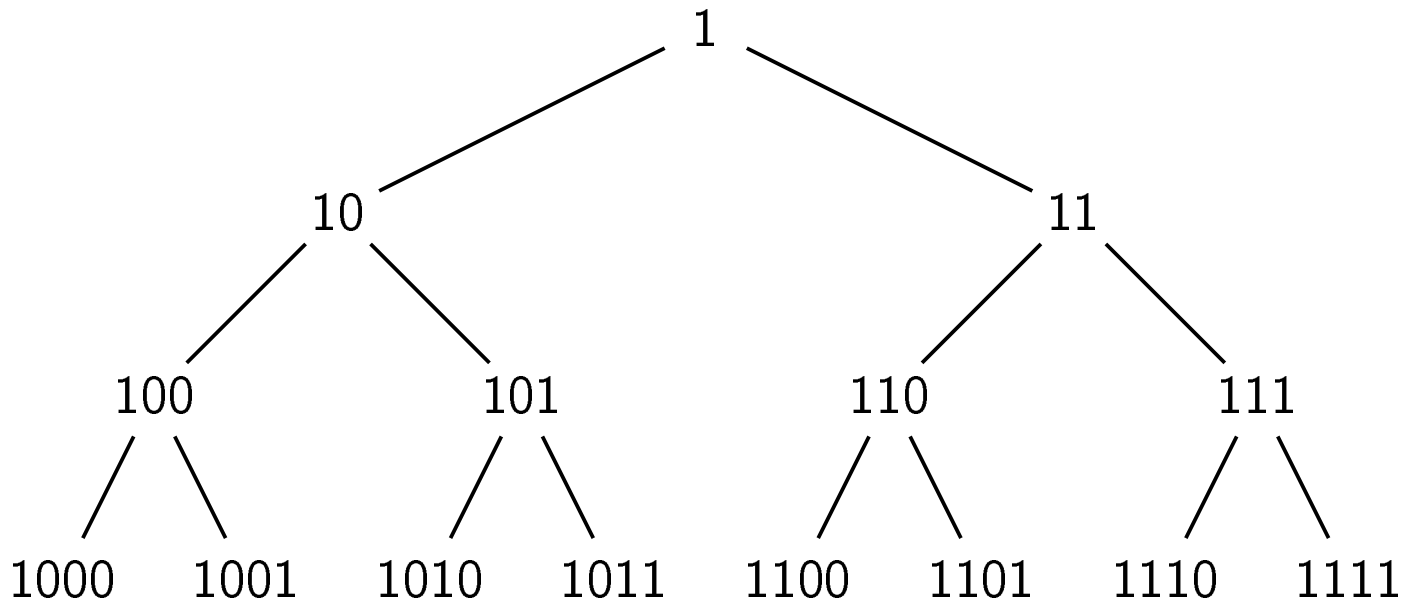
$\mathrm{LSB}(x) = x \,\&\, (-x)$? Hint: in 2's complement, $(-x) = \overline{x} + 1$.

Indexing trees

Indexing full binary trees



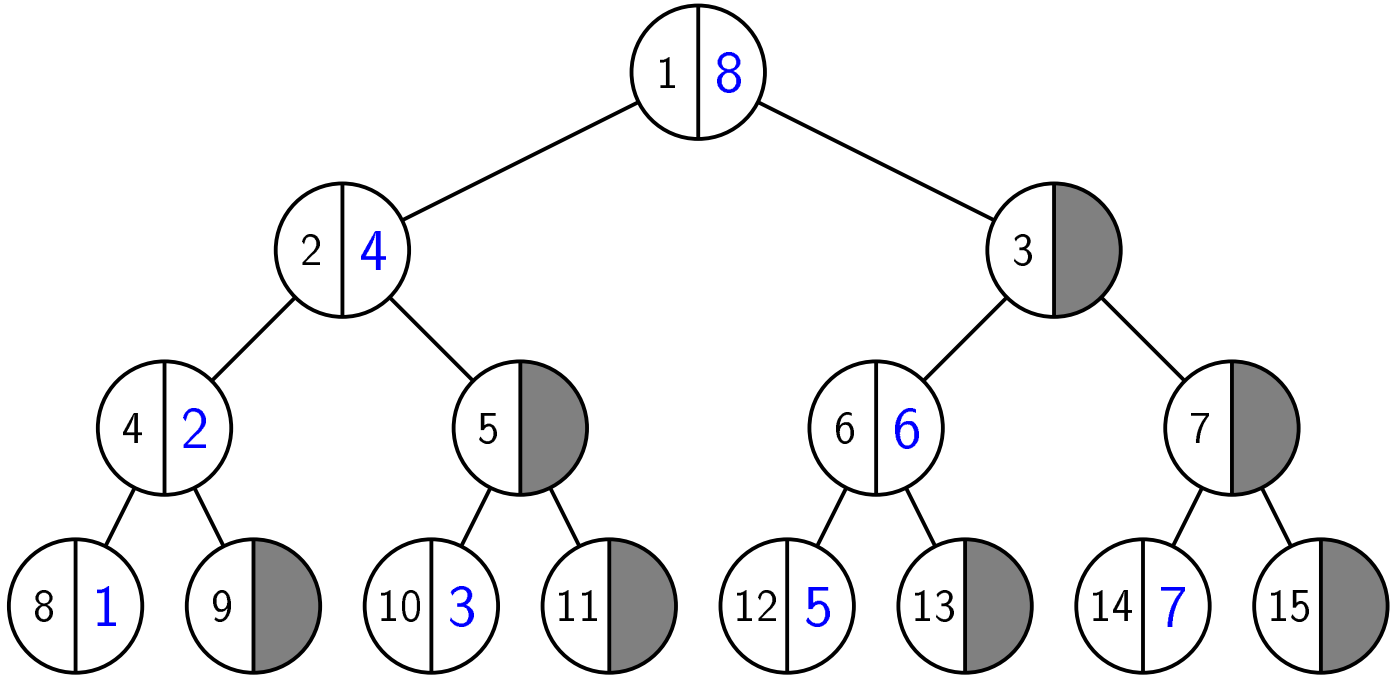
Indexing full binary trees, in binary



The Plan

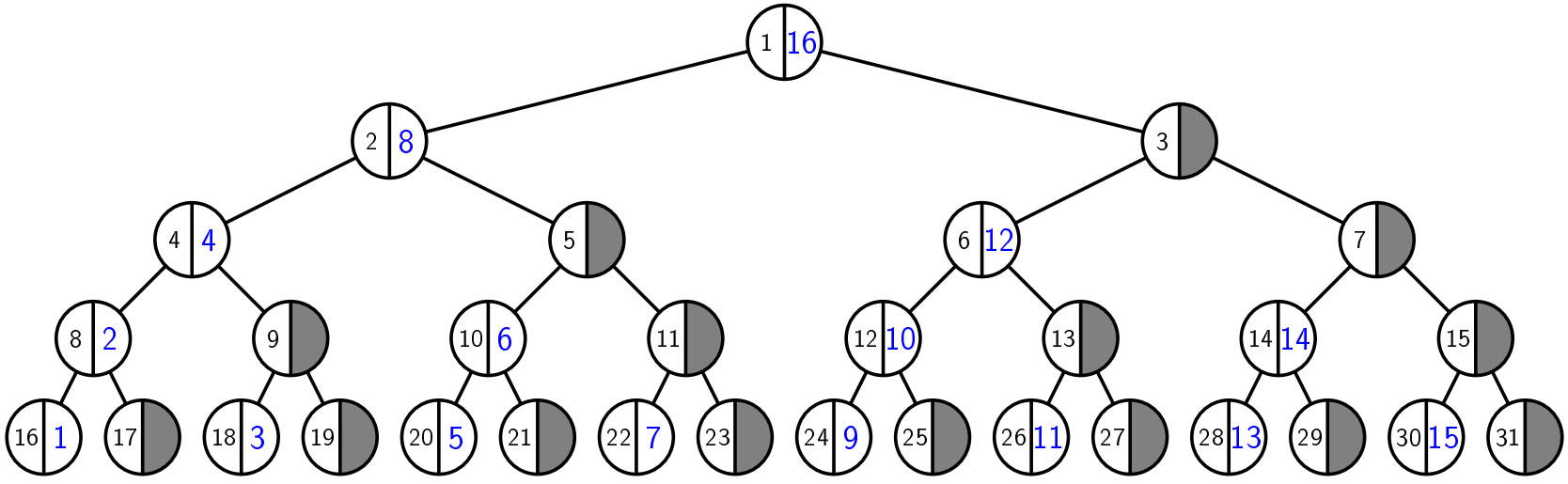
- Derive Fenwick tree $\leftrightarrow$ binary tree index conversion
- Compute motion through a Fenwick tree (array) as $\mathit{fromBinary} \circ \mathit{binaryMotion} \circ \mathit{toBinary}$
- Fuse
- Profit!

Fenwick → binary



1	2	3	4	5	6	7	8
8	4	10	2	12	6	14	1

Fenwick → binary



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
16	8	18	4	20	10	22	2	24	12	26	6	28	14	30	1

toBinary

$$[] \curlyvee \_ = []$$
$$(x :: xs) \curlyvee ys = x :: (ys \curlyvee xs)$$

$$s_0 = [1]$$
$$s_n = [2^n, 2^n + 2, \ldots, 2^{n+1} - 2] \curlyvee s_{n-1}$$

$$toBinary_n(k) = s_n \mathbin{!} k \qquad \text{(1-indexed)}$$

Interleaving lemmas

$$(xs \curlyvee ys) \,!\, 2k = ys \,!\, k$$
$$(xs \curlyvee ys) \,!\, (2k - 1) = xs \,!\, k$$

Simplifying toBinary

$$s_0 = [1]$$
$$s_n = [2^n, 2^n + 2, \ldots, 2^{n+1} - 2] \curlyvee s_{n-1}$$

$$\mathit{toBinary}_n(2k) = s_n \mathbin{!} 2k = s_{n-1} \mathbin{!} k$$
$$\mathit{toBinary}_n(2k-1) = [2^n, 2^n + 2, \ldots] \mathbin{!} k = 2^n + 2(k-1)$$

Simplifying toBinary

$$s_0 = [1]$$
$$s_n = [2^n, 2^n + 2, \ldots, 2^{n+1} - 2] \curlyvee s_{n-1}$$

$$toBinary_n(2k) = s_n \,!\, 2k = s_{n-1} \,!\, k$$
$$toBinary_n(2k-1) = [2^n, 2^n + 2, \ldots] \,!\, k = 2^n + 2(k-1)$$

$$toBinary_n(j) = \begin{cases} toBinary_{n-1}(j/2) & j \text{ even} \\ 2^n + j - 1 & j \text{ odd} \end{cases}$$

Simplifying toBinary

$$toBinary_n(j) = \begin{cases} toBinary_{n-1}(j/2) & j \text{ even} \\ 2^n + j - 1 & j \text{ odd} \end{cases}$$

Let $k = 2^a b$ where b is odd. Then

$$toBinary_n(2^a b) = toBinary_{n-a}(b) = 2^{n-a} + b - 1.$$

toBinary as bit operations?

$$toBinary_n(2^a b) = 2^{n-a} + b - 1$$

Given $k = 2^a b < 2^n$ ($2^n \to 1$ is special case):

- set 2^n bit $(2^n + 2^a b)$
- shift right until final bit is 1 $(2^{n-a} + b)$
- set final bit to 0 $(2^{n-a} + b - 1)$

$$toBinary_n = unset\ 0 \circ while\ even\ shr \circ set\ n$$

toBinary as bit operations?

$$toBinary_n(2^a b) = 2^{n-a} + b - 1$$

Given $k = 2^a b < 2^n$ ($2^n \to 1$ is special case):

- set 2^n bit $(2^n + 2^a b)$
- shift right until final bit is 1 $(2^{n-a} + b)$
- set final bit to 0 $(2^{n-a} + b - 1)$

$$toBinary_n = unset\ 0 \circ while\ even\ shr \circ set\ n$$

Question: what is the right DSL for expressing this bit manipulation procedure?

fromBinary

$$toBinary_n = unset\ 0 \circ while\ even\ shr \circ set\ n$$

$fromBinary_n$ ($1 \rightarrow 2^n$ is special case):

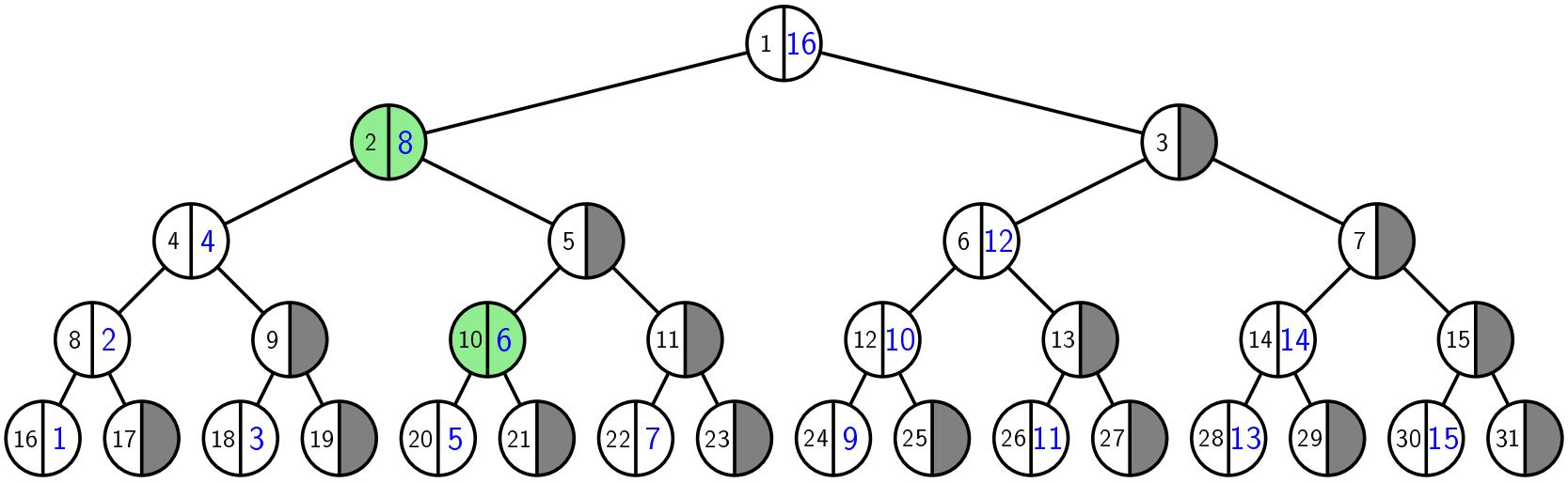
- set final bit to 1
- shift left until MSB is 2^n
- set 2^n bit to 0

$$fromBinary_n = unset\ n \circ while\ (< 2^n)\ shl \circ set\ 0$$

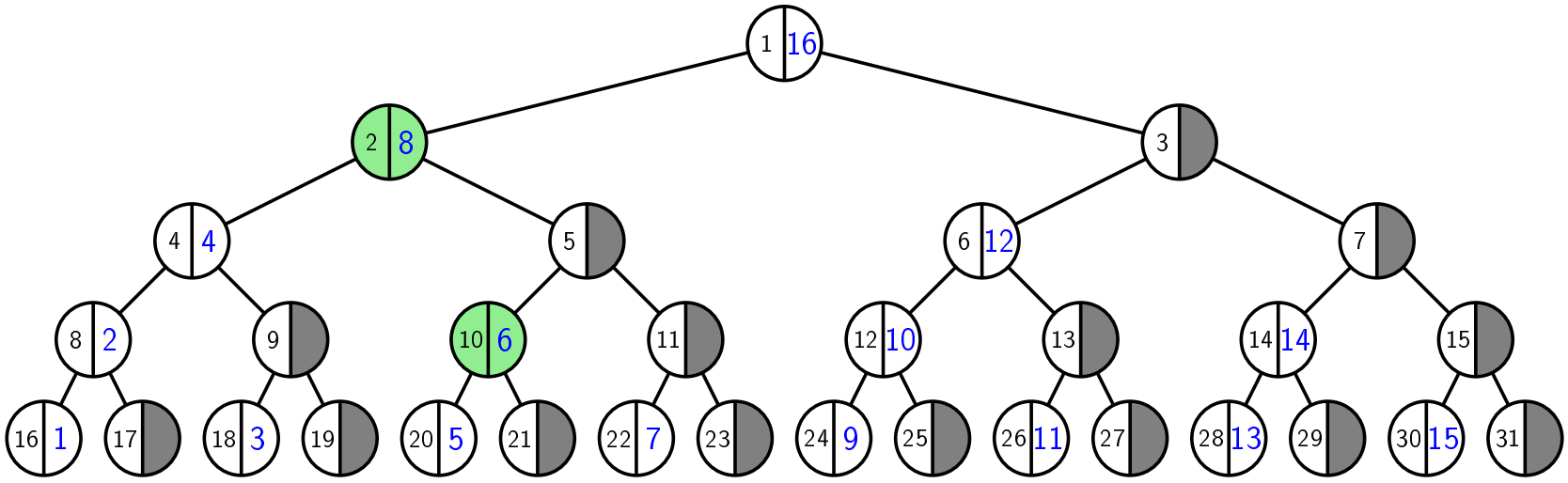
$fromBinary_n \circ toBinary_n$ example

$$
\begin{array}{ll}
 & 00110100 \\
\rightarrow & \{\quad set\ n \quad\} \\
 & 10110100 \\
\rightarrow & \{\quad while\ even\ shr \quad\} \\
 & 00101101 \\
\rightarrow & \{\quad unset\ 0 \quad\} \\
 & 00101100 \\
\rightarrow & \{\quad set\ 0 \quad\} \\
 & 00101101 \\
\rightarrow & \{\quad while\ (< 2^n)\ shl \quad\} \\
 & 10110100 \\
\rightarrow & \{\quad unset\ n \quad\} \\
 & 00110100
\end{array}
$$

Update


1 16
2 8
3
4 4
5
6 12
7
8 2
9
10 6
11
12 10
13
14 14
15
16 1
17
18 3
19
20 5
21
22 7
23
24 9
25
26 11
27
28 13
29
30 15
31

Update



$$activeParent = while\ odd\ shr \circ shr$$

Update

$$
\begin{aligned}
\mathit{nextUpdate}_n &= \mathit{fromBinary}_n \circ \mathit{activeParent} \circ \mathit{toBinary}_n \\
&= \mathit{unset}\ n \circ \mathit{while}\ (< 2^n)\ \mathit{shl} \circ \mathit{set}\ 0 \\
&\quad \circ \mathit{while}\ \mathit{odd}\ \mathit{shr} \circ \mathit{shr} \\
&\quad \circ \mathit{unset}\ 0 \circ \mathit{while}\ \mathit{even}\ \mathit{shr} \circ \mathit{set}\ n \\
&= \mathit{unset}\ n \circ \mathit{while}\ (< 2^n)\ \mathit{shl} \\
&\quad \circ \mathit{set}\ 0 \circ \mathit{while}\ \mathit{odd}\ \mathit{shr} \circ \mathit{while}\ \mathit{even}\ \mathit{shr} \\
&\quad \circ \mathit{set}\ n \\
&= \ldots\ ?\ \ldots \\
&= \lambda x.x + \mathit{LSB}\ x
\end{aligned}
$$

Update example

$$
\begin{array}{l}
\quad 00110 \\
\rightarrow \qquad \{ \quad \textit{set } n \quad \} \\
\quad 10110 \\
\rightarrow \qquad \{ \quad \textit{while even shr} \quad \} \\
\quad 01011 \\
\rightarrow \qquad \{ \quad \textit{while odd shr} \quad \} \\
\quad 00010 \\
\rightarrow \qquad \{ \quad \textit{set } 0 \quad \} \\
\quad 00011 \\
\rightarrow \qquad \{ \quad \textit{while } (< 2^n) \textit{ shl} \quad \} \\
\quad 11000 \\
\rightarrow \qquad \{ \quad \textit{unset } n \quad \} \\
\quad 01000
\end{array}
$$

Thanks!